

Platforms

An agent has to interact with the users through some communication channel. BAF's platforms wrap all the necessary code to build the connection between the agent and a communication channel.

All platforms implement the abstract class [Platform](#). It defines the methods any platform must have. So, if you would like to create your own platform (and contribute to the BAF development 😊), it must implement them according to the platform needs:

- [`initialize\(\)`](#): Initialize the platform. It is called internally when you run the agent (`Agent.run()`). You may need to set some things previously to the platform execution.
- [`start\(\)`](#): Start the platform execution. It is called internally when you run the agent (`Agent.run()`).
- [`stop\(\)`](#): Stop the platform. It is called internally when you stop the agent (`Agent.stop()`).
- [`send\(\)`](#): Send a payload (usually a message) to the user. This is a private method. It is called internally by other platform methods, such as the following one.
- [`reply\(\)`](#): Send a textual message to the user.

Additionally, a platform could have specific methods for it, as you can see in our implemented platforms.

Session persistence

BAF already persists user sessions at the backend level, so restarting an agent brings every user back to the last state with their stored parameters intact. However, the client UI must authenticate the user so the platform can map the incoming connection to the correct session.

- **Telegram** ships with its own user authentication, so persistence works out of the box as soon as you configure the Telegram platform. See [Telegram platform](#) for setup details.
- **WebSocket + Streamlit UI** includes a built-in login page that tags each browser session with a username before opening the socket. Learn how to enable persistence in [WebSocket platform](#).
- **Custom WebSocket interfaces** must implement the same identification protocol (authenticate the user and include the user identifier in each message) so the agent can reuse the persisted session. Follow the guidelines in [WebSocket platform](#) if you plan to build your own frontend on top of the WebSocket API.

WebSocket platform

The WebSocket Platform allows an agent to communicate with the users using the [WebSocket](#) bidirectional communications protocol.

This platform implements the WebSocket server, and it can establish connection with a client, allowing the bidirectional communication between server and client (i.e. sending and receiving messages).

The next figure shows how this connection works:

BAF comes with some User Interfaces (WebSocket clients) to use the WebSocket platform.

Of course, you are free to use or create your own UI as long as it has a WebSocket client that connects to the agent's WebSocket server.

- [Streamlit UI](#)
- [Chat widget](#)

(Their source code can be found in the `baf.platforms.websocket` package)

Note

There are some properties the agent needs in order to properly set the WebSocket connection. More details in the [configuration properties](#) documentation.

Restricting allowed origins

By default, the WebSocket server accepts connections from any origin. You can restrict which origins are allowed to connect by setting the `origins` property in your `config.yaml`:

```
platforms:
  websocket:
    host: localhost
    port: 8765
    origins:
      - "https://editor.besser-pearl.org"
      - "https://localhost:8000"
```

When `origins` is set, the server will reject WebSocket upgrade requests from any origin not in the list. When not set (default), all origins are accepted.

How to use it

After you instantiate your agent, simply call the following function:

```
agent = Agent('example_agent')
...
websocket_platform = agent.use_websocket_platform(use_ui=True)
```

If you don't want to use the UI we provide, simply set `use_ui` to `False`.

After that, you can use the platform to send different kinds of messages to the user (from [state bodies](#)):

- Text messages (strings):

```
websocket_platform.reply(session, 'Hello!')
```

- Text messages in [Markdown](#) format:

```
websocket_platform.reply_markdown(session, """
# Welcome to the agent experience
## Section 1
- one
- two
""")
```

- Text messages in [HTML](#) format:

```
websocket_platform.reply_html(session, """
<h1>Title</h1>
<ul>
  <li>Apples</li>
  <li>Bananas</li>
  <li>Cherries</li>
</ul>
""")
```

- Pandas [DataFrames](#):

```
websocket_platform.reply_dataframe(session, df)
```

- List of options (buttons): Display a list of options as buttons and let the user choose one

```
websocket.reply_options(session, ['Yes', 'No'])
```

- Plotly [figure](#):

```
websocket_platform.reply_plotly(session, plot)
```

- Files:

```
file = File(file_name="name", file_type="type", file_base64="file_base64")
websocket_platform.reply_file(session, file)
```

- Locations:

```
latitude, longitude = 49.50177449302207, 5.94862573528648
websocket_platform.reply_location(session, latitude, longitude)
```

- [RAG](#) Messages:

```
rag_message: RAGMessage = session.run_rag()
websocket_platform.reply_rag(session, rag_message)
```



We are working on other replies (files, media, charts...). They will be available soon, stay tuned!

The WebSocket platform allows the following kinds of user messages:

- Text messages
- Voice messages
- Files

Enabling persistent user sessions

When building your own UI on top of the WebSocket API, implement user authentication so every connection can be tied to a stable identifier. BAF maps connections to sessions using either the X-User-ID header or the `user_id` query parameter from the handshake URL. If both are provided, the header takes precedence. If neither identifier is available, the platform treats the connection as a new anonymous user.

Once your client authenticates users, include the identifier in the WebSocket handshake headers:

```
ws = websocket.WebSocketApp(
    f"ws://{host}:{port}/",
    header={"X-User-ID": user_id},
    on_open=on_open,
    on_message=on_message,
    on_error=on_error,
    on_close=on_close,
    on_ping=on_ping,
    on_pong=on_pong,
)
```

If you cannot control the HTTP headers, you can also attach the identifier as a query parameter on the WebSocket URL:

```
ws = websocket.WebSocketApp(
    f"ws://{host}:{port}/?user_id={user_id}",
    on_open=on_open,
    on_message=on_message,
    on_error=on_error,
    on_close=on_close,
    on_ping=on_ping,
    on_pong=on_pong,
)
```

On the agent's side, you'll need to start the monitoring database and set the `persist_sessions=True` parameter when initializing the agent:

```
agent = Agent('example_agent', persist_sessions=True)
```

Requesting the chat history for a session

If your UI client wants to request the chat history for a given session (for example, when a user reconnects after a disconnection), it can do so by sending a message with the action `FETCH_USER_MESSAGES` and the corresponding `user_id` in the payload:

```
payload = Payload(
    action=PayloadAction.FETCH_USER_MESSAGES,
    message=None,
    user_id=username,
)
```

The websocket platform will start by sending the previous messages to the client with a flag "history" set to True, so the client can differentiate between historical messages and new incoming messages.

Communication between agents: Multi-agent systems

An agent can communicate, not only with human users, but with other agents as well. When we design such agents they become a **multi-agent system**.

A multi-agent or agentic system is an approach to solve complex problems by defining multiple agents to tackle specific tasks. These tasks divide the main problem into smaller and easier to handle problems. With these systems, we get:

- **Easier agentic design:** divide your problem into small tasks and create a simpler agent for each of them.
- **Modular design:** each agent acts as an independent actor and can be part of many agentic systems, being called from and when necessary.

The communication between agents is done thanks to the WebSocketPlatform. An agent *A* can send a message to another agent *B* by simply connecting to agent *B*'s WebSocket platform:

```
# Agent A
def get_weather_body(session: Session):
    # Send message to Agent B, providing its WebSocketPlatform address and the
    message
    session.send_message_to_websocket('ws://localhost:6000', session.message)
```

Agent *B* will receive the message and will treat it the same way as if it was a human message. It will create a new session for agent *A*, detect the intent, transition to another state, etc.

```
# Agent B, let's assume this state is run when it receives agent A's message
def generate_weather_body(session: Session):
    # Some code to get the weather from the message agent A sent
    temperature = api.get_weather(session.message)
    # Send the temperature to the session's user (i.e., agent A)
    session.reply(temperature)
```

After that, agent *A* will receive agent *B*'s message (the temperature), which will trigger intent classification, transition to another state, etc.

```
# Agent A's next state
def show_weather_body(session: Session):
    # Just print what Agent B sent, i.e., the temperature
    session.reply(session.message)
```

Telegram platform

The Telegram Platform allows an agent to communicate with the users using [Telegram](#).

Telegram is a great platform for chatbots/agents. You can create a Telegram bot through the [Bot Father](#) and then, link it to our BAF agent. This way, you can define your agent architecture and logics, and then use your Telegram bot as the communication channel.

Our Telegram Platform uses the [python-telegram-bot](#) library, which is a Telegram Bot API wrapper for Python.

Note

There are some properties the agent needs in order to properly set the Telegram connection. More details in the [configuration properties](#) documentation.

How to use it

After you instantiate your agent, simply call the following function:

```
agent = Agent('example_bot')
...
telegram_platform = agent.use_telegram_platform()
```

After that, you can use the platform to send different kinds of messages to the user (from [state bodies](#)):

- Text messages (strings):

```
telegram_platform.reply(session, 'Hello!')
```

- Files:

```
file = File(file_name="name", file_type="type", file_base64="file_base64")
telegram_platform.reply_file(session, file, message='Optional caption')
```

- Images:

```
telegram_platform.reply_image(file=f, session=session, message='Optional caption') # the file must be an image
```

- Locations:

```
latitude, longitude = 49.50177449302207, 5.94862573528648
telegram_platform.reply_location(session, latitude, longitude)
```

Note

The agent cannot detect when a user opens the Telegram chat window. Therefore, to start the conversation, it is needed a first message to “wake the agent up”. After that, it will start running the initial state.

 We are working on other replies (files, media, charts...). They will be available soon, stay tuned!

Handlers

The python-telegram-bot library (Telegram Bot API Python wrapper under our Telegram Platform) uses [handlers](#) to handle bot updates (e.g., the user sends a text message, an image, a command, etc.). Our Telegram Platform interface allows you to add custom handlers to your agent, so this feature is not lost when using BAF.

This is an example handler function that will be executed when the agent receives the `/help` command:

```

from telegram import Update
from telegram.ext import CommandHandler, ContextTypes
...
async def help(update: Update, context: ContextTypes.DEFAULT_TYPE):
    session_id = str(update.effective_chat.id)
    session = agent.get_or_create_session(session_id)
    session.reply('Please introduce a number between 1 and 10')

help_handler = CommandHandler('help', help)
telegram_platform.add_handler(help_handler)

```

Note

You can take the built-in handlers as a reference to create handlers integrated with the agent.

Our Telegram Platform has the following built in handlers:

- One to handle all user text messages (that simply captures the messages and sends them to the agent).
- A *reset* handler that resets the agent when the user writes the command `/reset`.
- A voice message handler (you need to set the [Speech-to-Text](#) component to enable voice messages)
- A file handler to receive files from the user
- An image handler to receive images from the user (images are a subset of files)

Telegram Wrapper

The BAF Telegram Platform wraps some functionalities of the `python-telegram-bot` library (such as adding handlers or replying messages, files or locations), but not all of them.

In order to use other features not included in BAF yet, we included a `__getattr__` function in the `TelegramPlatform` class. It forwards the method calls not implemented in `TelegramPlatform` to the underlying Telegram bot ([ExtBot](#) class, which is an extension of the [Bot](#) class).

That means you can call any function from the TelegramPlatform as you would do in the Telegram bot!

Let's see an example.

You could use [send_audio](#) to send audios to the user. Since this is not integrated in our `TelegramPlatform`, you can simply call it and it will be forwarded:

```

def example_body(session: Session):
    # The session id is the Telegram chat_id
    telegram_platform.send_audio(session.id, my_audio, title='Hello World')

```

Note that the `TelegramPlatform` wrappers also involve other actions. For instance, when the agent replies a message, it can be added to a chat history [database](#). You can also customize what is done when calling any function. You could update the chat history to record the audio messages, either adding the audio or simply a log message:

```

from datetime import datetime
from baf.core.message import Message, MessageType

```

```
def custom_send_audio(session: Session, audio):
    message = Message(MessageType.AUDIO, audio, is_user=False,
timestamp=datetime.now())
    session.save_message(message)
    telegram_platform.send_audio(session.id, audio, title='Hello World')

def example_body(session: Session):
    custom_send_audio(session, audio)
```

Enabling session persistence

The Telegram platform has session persistence enabled by default if the [monitoring database](#) is running.

GitHub platform

The GitHub Platform allows an agent to receive events from [GitHub's webhooks](#) and perform actions on repositories through the [GitHub API](#).

Our GitHub Platform uses the [gidgethub](#) library, which is an asynchronous GitHub API wrapper for Python.

Note

There are some properties the agent needs in order to properly set GitHub and webhook connections. More details in the [configuration properties](#) documentation.

How to use it

After you instantiate your agent, simply call the following function:

```
agent = Agent('example_agent')
...
github_platform = agent.use_github_platform()
```

After that, you can use the different [events sent by GitHub](#) to trigger transitions in your agent.

```
# Pull Request
idle.when_event(PullRequestOpened()).go_to(pull_state)
# Issues
idle.when_event(IssuesOpened()).go_to(issue_state)
# Labels
idle.when_event(LabelCreated()).go_to(label_state)
# Stars
idle.when_event(StarCreated()).go_to(star_state)
```

Note

The agent needs to provide a public URL to receive the webhooks. For local testing you can use [ngrok](#)

In addition to webhooks events, the github platform offers:

- Access to the payload of the received event
- Wrapper classes on top of the issues and users payload
- Methods to open, get, comment, label and assign a user to an issue

These abstractions allows to receive webhooks events and perform agent actions on the repository as a reaction. The following example wait for issues to open to add a thanking message as comment:

```
# How to import the Issue class
from baf.platforms.github.github_objects import Issue

def issue_body(session: Session):
    # Access through the Session to the IssuesOpened GitHubEvent that triggered
    the transition
    event: GitHubEvent = session.event
    # Wrap the issue object of the payload in our abstraction
    issue = Issue(event.payload['issue'])
    # Add a thanking message to the opened issue
    github_platform.comment_issue(issue
        f'Hey,\n\nThanks for opening an issue {issue.creator.login}!<br>We will
        look at that as soon as possible.')
```

 We are working on providing abstractions for more concepts than issues, so stay tuned!

Gidgethub Wrapper

The BAF GitHub Platform wraps some functionalities of the gidgethub library (such as handling webhooks or act on issues), but not all of them.

In order to use other features not included in BAF yet, we included a `__getattr__` function in the GitHubPlatform class. It forwards the method calls not implemented in GitHubPlatform to the underlying GitHubAPI ([gidgethub.aihttp.GitHubAPI](#) class, which is an extension of the abstract [gidgethub.abc.GitHubAPI](#) class).

That means you can call any function from the GitHubPlatform as you would do in the GitHubAPI!

Let's see an example.

You could use [getitem](#) to get the list of contributors to a repository. Since this is not integrated in our GitHubPlatform, you can simply call it and it will be forwarded:

```
def example_body(session: Session):
    payload = github_platform.getitem(f'/repos/OWNER/REPO/contributors')
```

GitLab platform

The GitLab Platform allows an agent to receive events from [GitLab's webhooks](#) and perform actions on repositories through the [GitLab API](#).

Our GitLab Platform uses the [gidgetlab](#) library, which is an asynchronous GitLab API wrapper for Python.

Note

There are some properties the agent needs in order to properly set GitLab and webhook connections. More details in the [configuration properties](#) documentation.

How to use it

After you instantiate your agent, simply call the following function:

```
agent = Agent('example_agent')
...
gitlab_platform = agent.use_gitlab_platform()
```

After that, you can use the different [events sent by GitLab](#) to trigger transitions in your agent.

```
# Importing the events
from baf.platforms.gitlab.gitlab_webhooks_events import MergeRequestOpened,
IssuesOpened, WikiPageCreated, Push

# Merge Request
idle.when_event(MergeRequestOpened()).go_to(merge_state)
# Issues
idle.when_event(IssuesOpened()).go_to(issue_state)
# Wiki Pages
idle.when_event(WikiPageCreated()).go_to(wiki_state)
# Push
idle.when_event(Push()).go_to(push_state)
```

Note

The agent needs to provide a public URL to receive the webhooks. For local testing you can use [ngrok](#).

In addition to webhooks events, the gitlab platform offers:

- Access to the payload of the received event
- Wrapper classes on top of the issues and users payload
- Methods to open, get, comment, label and assign a user to an issue

These abstractions allows to receive webhooks events and perform agent actions on the repository as a reaction. The following example wait for issues opening events to add a thanking message as comment:

```
# How to import the Issue class
from baf.platforms.gitlab.gitlab_objects import Issue

def issue_body(session: Session):
    # Access through the Session to the IssuesOpened GitLabEvent that triggered
    the transition
    event: GitLabEvent = session.event
    # Extract useful information
    user_repo = event.payload['project']['path_with_namespace'].split('/')
    issue_iid = event.payload['object_attributes']['iid']
    # Get the issue as object
    issue: Issue = gitlab_platform.get_issue(
```

```

        user=user_repo[0],
        repository=user_repo[1],
        issue_number=issue_iid)
# Add a thanking message to the opened issue
gitlab_platform.comment_issue(issue,
    'Hey,\n\nThanks for opening an issue!<br>We will look at that as soon as
possible.')
```

Gitgetlab Wrapper

The BAF GitLab Platform wraps some functionalities of the gidgetlab library (such as handling webhooks or act on issues), but not all of them.

In order to use other features not included in BAF yet, we included a `__getattr__` function in the `GitLabPlatform` class. It forwards the method calls not implemented in `GitLabPlatform` to the underlying `GitLabAPI` ([gidgetlab.aiohttp.GitLabAPI](#) class, which is an extension of the abstract [gidgetlab.abc.GitLabAPI](#) class).

That means you can call any function from the `GitLabPlatform` as you would do in the `GitLabAPI`!

Let's see an example.

You could use [getitem](#) to get the list of contributors to a repository. Since this is not integrated in our `GitLabPlatform`, you can simply call it and it will be forwarded:

```
def example_body(session: Session):
    payload =
gitlab_platform.getitem(f'/projects/NAMESPACE%2FPROJECT_PATH/repository/
contributors')
```

Note

Here we use the [Namespaced path](#) notation (i.e. “`NAMESPACE%2FPROJECT_PATH`”) for the project, but using project id works too.

A2A platform

A2A (Agent-to-Agent) is a platform that follows Google's [A2A protocol](#) and allows direct communication between agents developed using BESSER-Agent-ic-Framework and other frameworks too.

How to use it

Required imports are:

```
import asyncio
from aiohttp import web
from baf.core.agent import Agent
from baf.platforms.a2a.agent_registry import AgentRegistry
from baf.platforms.a2a.server import create_app
```

Define an agent (with its name), attach it to an A2A platform and register the agent's platform with an ID (this is the agent ID that the users will use to identify this agent).

```
registry = AgentRegistry()
agent = Agent('TestAgent')
a2a_platform = agent.use_a2a_platform()
registry.register('test_platform', a2a_platform)
```

Define an asynchronous method that the agent has to execute (the functionality of agent) and register it in the corresponding agent's platform's router with your desired ID (this is the method ID that the users will use to invoke this method).

```
async def my_method(method_params: dict):
    # Logic of this method

a2a_platform.router.register("method_id", my_method)
```

You can add the agent's metadata using the built-in helper functions.

```
a2a_platform.add_capabilities(['capability1', 'capability2'])
a2a_platform.add_descriptions(['description1', 'description2'])
a2a_platform.add_examples([{'first example': 'example1', 'second example':
'example2'}])
```

To add agent specific methods to agent's metadata, either an automated built-in function (automatically identify and add methods that are registered with the platform's router) or the helper function to add it manually can be used.

```
a2a_platform.populate_methods_from_router() # method description will be taken
from method's doc string
```

or

```
a2a_platform.add_methods([{"name": "my_method", "description": "method
description"}])
```

Execute the platform on port=8000 using the following lines. Registry has the platform (agent(s)) and platform has the method(s) corresponding to the agent(s).

```
app = create_app(registry=registry)
web.run_app(app, port=8000)
```

Executing this script will provide a message similar to the following in your terminal window.

```
<timestamp> - BESSER Agentic Framework - INFO - Registering agent TestAgent
===== Running on http://0.0.0.0:8000 =====
(Press CTRL+C to quit)
```

In a browser, you can see the agents hosted on your server at the endpoint:

<http://localhost:8000/agents>. Each agent will have its ID, name, description, capabilities and a link to its agent card.

In a browser, you can see the agent card of your agent at the endpoint:

http://localhost:8000/agents/test_platform/agent-card. test_platform is our agent_id. For getting the agent card of another agent, test_platform should be replaced with the corresponding agent's agent_id.

Note

Methods corresponding to the following method IDs are by default included in all the agents:

method_id: task_create, params: method, params - creates a new synchronous/asynchronous task (for executing the method given in params) and adds it to the task dictionary.

method_id: create_task_and_run, params: method, params - creates a new asynchronous task (for executing the method given in params), adds it to the task dictionary, sets task's status to PENDING and executes the method provided within params. When the execution starts, corresponding task's status will be changed to RUNNING and once the execution is finished, the status will be changed to DONE. In case of an error, its status will be changed to ERROR.

method_id: task_status, params: task_id - provides the current status of the task (PENDING, RUNNING, DONE/ERROR) based on its agent_id and task_id.

In a different terminal, you can use the agent present on your server hosted at <http://localhost:8000/a2a> endpoint. Here is an example CURL command for using the agent.

To send a message to the server from your system (in a different terminal than the one that is executing your script)

```
curl -X POST http://localhost:8000/a2a \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0", "agent_id": "test_platform", "method":
"create_task_and_run", "params":{"method":"method_id", "params":
{method_params:<param_key>}}, "id":1}'
```

There are three ways to get the status of the task that is being executed in the agent:

1. by CURL command in the terminal
2. In browser via HTTP polling.
3. In browser via SSE. You can watch the result/status live, no need to refresh the page.

```
curl -X POST http://localhost:8000/a2a \
  -H "Content-Type: application/json" \
  -d '{"jsonrpc":"2.0", "agent_id": "test_platform", "method": "task_status",
"params":{"task_id":<task_id>}, "id":2}'
```

or

Open in browser: http://localhost:8000/agents/test_platform/tasks

or

Open in browser: http://localhost:8000/agents/test_platform/events/<task_id>

Note

The above mentioned example is for executing a single agent. There are multiple ways this platform can be used. For three agents named A, B and C, the following are also possible.

Executing agent A or B or C (same as above)

Decentralized orchestration: Using agent A to invoke agent B

Parallel execution: A || B || ...

Sequential execution: $A \rightarrow B \rightarrow \dots$

Hybrid execution: $A \parallel B \rightarrow C \dots$

More examples can be viewed at [A2A multiagent execution examples](#)